

ΑΝΤΙΚΕΙΜΕΝΟΣΤΡΕΦΗΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ ΜΕ C++

Προετοιμασία για το εργαστήριο 1

Μεταγλώττιση με g++, στάδια μεταγλώττισης, namespaces, πέρασμα παραμέτρων και static τοπικές μεταβλητές

`-o -Wall -Werror -std`

`preprocess → link`

`int x vs int &x`

`static int s`

ΑΠΟ ΤΟ .CPP ΣΤΟ ΕΚΤΕΛΕΣΙΜΟ

Τα τέσσερα στάδια, με αυτή τη σειρά

- | | | | |
|---|---------------------------------------|--------------------------------------|---|
| 1 | Προεπεξεργασία
preprocessor | <code>prog.cpp → prog.i</code> | Εκτελεί τις οδηγίες <code>#include</code> και <code>#define</code> , αφαιρεί σχόλια |
| 2 | Μεταγλώττιση
compiler | <code>prog.i → prog.s</code> | Ελέγχει σύνταξη και τύπους, παράγει assembly |
| 3 | Συμβολομετάφραση
assembler | <code>prog.s → prog.o</code> | Μετατρέπει το assembly σε κώδικα μηχανής (object file) |
| 4 | Σύνδεση
linker | <code>*.o + βιβλιοθήκες → app</code> | Ενώνει τα object files και τις βιβλιοθήκες σε εκτελέσιμο |

Δοκιμάστε: `g++ -E` σταματά μετά την προεπεξεργασία · `g++ -S` μετά τη μεταγλώττιση · `g++ -c` μετά τη συμβολομετάφραση

ΑΠΟ ΤΟ .CPP ΣΤΟ ΕΚΤΕΛΕΣΙΜΟ

Τα switches του g++ που πρέπει να ξέρετε

Switch	Τι κάνει	Προσοχή
-o calc	Ορίζει το όνομα του αρχείου εξόδου: calc (calc.exe στα Windows)	Δεν εκτελεί το πρόγραμμα· χωρίς -o βγαίνει a.out / a.exe
-Wall	Ενεργοποιεί ένα ευρύ σύνολο συνηθισμένων προειδοποιήσεων (π.χ. μεταβλητή που δεν χρησιμοποιείται)	Δεν ανάβει όλες τις προειδοποιήσεις: το -Wextra προσθέτει κι άλλες
-Werror	Αντιμετωπίζει κάθε προειδοποίηση ως σφάλμα	Η μεταγλώττιση αποτυγχάνει, δεν παράγεται εκτελέσιμο
-std=c++20	Ορίζει το πρότυπο (έκδοση) της C++ για τη μεταγλώττιση	Επιλογές: c++11, c++14, c++17, c++20

ΑΠΟ ΤΟ .CPP ΣΤΟ ΕΚΤΕΛΕΣΙΜΟ

Ένα αρχείο, τρεις εντολές, τρία αποτελέσματα

sq.cpp

```
int square(int n) {  
    int result = 0; // περιττή  
    return n * n;  
}  
int main() {  
    return square(3);  
}
```

```
g++ sq.cpp -o sq
```

Μεταγλωττίζεται σιωπηλά, δημιουργείται το sq

```
g++ -Wall sq.cpp -o sq
```

Προειδοποίηση «unused variable 'result'», αλλά το sq δημιουργείται κανονικά

```
g++ -Wall -Werror sq.cpp -o sq
```

Αποτυγχάνει: η προειδοποίηση για το result γίνεται σφάλμα, δεν βγαίνει εκτελέσιμο

ΧΩΡΟΙ ΟΝΟΜΑΤΩΝ

Το `#include` φέρνει τις δηλώσεις, το `using` συντομεύει τα ονόματα

```
#include <iostream>
#include <string>
int main() {
    std::string s = "C++";
    std::cout << s;
}
```

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s = "C++";
    cout << s;
}
```

std::

Ο χώρος ονομάτων της standard βιβλιοθήκης: εκεί ζουν τα `cout`, `cin`, `string`, `vector`.

using namespace std;

Τα ονόματα του `std` γράφονται χωρίς πρόθεμα. Δεν συμπεριλαμβάνει κανένα αρχείο.

#include <...>

`<iostream>` για `cout` και `cin`, `<string>` για `string`: κάθε εργαλείο έχει την επικεφαλίδα του.

ΠΈΡΑΣΜΑ ΠΑΡΑΜΈΤΡΩΝ

Με τιμή ή με αναφορά: αντίγραφο ή ψευδώνυμο

ΜΕ ΤΙΜΗ

void show(double v)

- Το v είναι αντίγραφο του ορίσματος, τοπική μεταβλητή της συνάρτησης
- Οι αλλαγές στο v δεν φτάνουν στον καλούντα
- Η συνάρτηση μπορεί να αλλάξει το v, αλλά αλλάζει μόνο το αντίγραφο
- Δέχεται και σταθερές: show(2.5) μεταγλωττίζεται

ΜΕ ΑΝΑΦΟΡΑ

void scale(double &v)

- Το v είναι ψευδώνυμο (alias) του ορίσματος, χωρίς αντιγραφή
- Οι αλλαγές στο v φαίνονται στον καλούντα
- Μένει δεμένο στην ίδια μεταβλητή σε όλη την κλήση
- Θέλει μεταβλητή: scale(2.5) δίνει σφάλμα μεταγλώττισης

Η αναφορά δουλεύει με οποιονδήποτε τύπο: int, double, string, πίνακες, αντικείμενα.

ΠΈΡΑΣΜΑ ΠΑΡΑΜΈΤΡΩΝ

Η αναφορά είναι ένα δεύτερο όνομα για την ίδια μεταβλητή

```
int score = 7;  
int &s = score; // s: άλλο όνομα του score  
s = s + 3;      // ίδιο με score = score + 3  
cout << score; // 10
```

score, s →

10

μία θέση μνήμης, δύο
ονόματα

Δένεται στη δήλωση

Σκέτο `int &s`; είναι σφάλμα· γράφουμε `int &s = score`;

Δεν αλλάζει στόχο

Το `s = other`; αντιγράφει την τιμή του `other` στο `score`· το `s` μένει δεμένο στο `score`

Ως παράμετρος

Η συνάρτηση δουλεύει πάνω στη μεταβλητή του καλούντα, χωρίς αντίγραφο

Με τιμή, ο καλών δεν βλέπει καμία αλλαγή

```
void reset(int n) { n = 0; }  
int main() {  
    int k = 8;  
    reset(k);  
    cout << k;  
}
```

Εμφανίζει

8

Το n είναι αντίγραφο του k: μηδενίζεται και χάνεται με την επιστροφή.

```
void order(int a, int b) {  
    if (a > b) { int t = a; a = b; b = t; }  
}  
int main() {  
    int x = 9, y = 4;  
    order(x, y);  
    cout << x << " " << y;  
}
```

Εμφανίζει

9 4

Ταξινομούνται μόνο τα αντίγραφα. Με `int &a, int &b` θα τύπωνε 4 9.

Μικτό πέρασμα: αλλάζει μόνο ό,τι έχει &

```
void h(int &x, int y) {  
    x = x * y;  y = 1;  
}  
int main() {  
    int a = 2, b = 5;  
    h(a, b);  
    cout << a << " " << b;  
}
```

Εμφανίζει **10 5**

$x \equiv a$ (αναφορά), $y = 5$ (αντίγραφο)

$x = 2 * 5 \rightarrow$ το a γίνεται 10

$y = 1$ αλλάζει μόνο το αντίγραφο $\rightarrow b = 5$

```
void k(int a, int &b) {  
    a = a * 2;  b = b - a;  
}  
int main() {  
    int x = 3, y = 20;  
    k(x, y);  
    cout << x << " " << y;  
}
```

Εμφανίζει **3 14**

$a = 3$ (αντίγραφο), $b \equiv y$ (αναφορά)

$a = 3 * 2 = 6 \rightarrow$ το x μένει 3

$b = 20 - 6 = 14 \rightarrow$ το y γίνεται 14

ΠΈΡΑΣΜΑ ΠΑΡΑΜΈΤΡΩΝ

Δύο αποτελέσματα; Παράμετροι εξόδου με αναφορά

```
void divide(int a, int b, int &q, int &r) {  
    q = a / b;    // πηλίκο  
    r = a % b;    // υπόλοιπο  
}  
int quot, rem;  
divide(17, 5, quot, rem);    // quot = 3, rem = 2
```

Πρωτότυπο	Αξιολόγηση
int divide(int a, int b);	Λάθος: επιστρέφει μόνο μία τιμή
void divide(int a, int b, int &q, int &r);	Σωστό: τα q, r γράφονται στον καλούντα
void divide(int &a, int &b, int q, int r);	Λάθος: τα αποτελέσματα μένουν σε αντίγραφα
void divide(int a, int b, int q, int r);	Λάθος: όλα αντίγραφα, χάνονται

Δύο σφάλματα μεταγλώττισης με αναφορές

Μη-const αναφορά σε σταθερά

```
void twice(int &x) { x *= 2; }  
twice(3);    // σφάλμα
```

Μια `int&` πρέπει να δένεται σε μεταβλητή· το 3 δεν έχει θέση που να αλλάξει.

Σωστά: `int n = 3; twice(n);` → `n = 6`

Αλλαγή μέσω `const` αναφοράς

```
void show(const int &x) {  
    x++;    // σφάλμα  
}
```

Το `const` απαγορεύει κάθε αλλαγή μέσω της αναφοράς.

Χρήση: διάβασμα χωρίς αντιγραφή, π.χ. `const string &name`

Αντίθετα, μια `const int&` δέχεται και σταθερές: `void print(const int &x); print(7);` μεταγλωττίζεται.

STATIC ΤΟΠΙΚΈΣ ΜΕΤΑΒΛΗΤΈΣ

Τέσσερις ιδιότητες μιας static τοπικής μεταβλητής

Αρχικοποιείται μία φορά

Την πρώτη φορά που η εκτέλεση φτάνει στη δήλωση, όσες κλήσεις κι αν ακολουθήσουν.

Κρατά την τιμή της

Ζει σε όλη τη διάρκεια του προγράμματος, όχι στη στοίβα· δεν χάνεται στην επιστροφή.

Ορατή μόνο στη συνάρτηση

Η εμβέλεια μένει τοπική: οι άλλες συναρτήσεις του αρχείου δεν τη βλέπουν.

Χωρίς αρχικοποίηση: 0

Το `static int count;` ξεκινά από 0. Μια απλή τοπική `int count;` έχει απροσδιόριστη τιμή.

`static` δεν σημαίνει `const`: η τιμή της αλλάζει κανονικά.

Η γραμμή αρχικοποίησης τρέχει μόνο την πρώτη φορά

```
int next(int start) {
    static int n = start;
    n += 2;
    return n;
}

int main() {
    cout << next(1) << " ";
    cout << next(100);
}
```

Κλήση	Αρχικοποίηση	n += 2 → return
next(1)	πρώτη φορά: n = 1	n = 3 → 3
next(100)	παραλείπεται (n = 3)	n = 5 → 5

Εμφανίζει **3 5**

Το όρισμα 100 της δεύτερης κλήσης δεν φτάνει ποτέ στο n.

Απλή τοπική και static, δίπλα δίπλα

```
void tick() {
    int x = 0;
    static int s = 0;
    x += 2; s += 2;
    cout << x << "-" << s << " ";
}
int main() { tick(); tick(); tick(); }
```

Κλήση	x	s	Τυπώνει
1η	2	2	2-2
2η	2	4	2-4
3η	2	6	2-6

Εμφανίζει

2-2 2-4 2-6

Το x ξαναγεννιέται με 0 σε κάθε κλήση· το s συνεχίζει από εκεί που έμεινε.